KỸ THUẬT LẬP TRÌNH C#

Chương 1: Giới thiệu về Máy tính, Internet và Visual C#

BÀI GIẢNG CHUẨN ĐẠI HỌC (3 TIẾT TÍCH HỢP)
Giáo trình: *Visual C# How to Program* (Deitel & Deitel)

Khoa Công nghệ Thông tin – Đại học Đông Á
Đối tượng: Lớp IT26M (Khóa 2026)

Học kỳ I – Năm học 2026

Bản quyền bài giảng thuộc Bộ môn Kỹ thuật Lập trình – Phục vụ đào tạo Đại học chính quy

Mục tiêu Bài học & Chuẩn đầu ra (CLOs)

Chuẩn đầu ra sinh viên cần đạt được sau buổi học 3 tiết:

- **CLO 1.1 (Understand):** Giải thích kiến trúc phần cứng, định luật Moore, 6 khối chức năng logic của máy tính và cấu trúc phân cấp dữ liệu từ Bit $\rightarrow$ Database $\rightarrow$ Big Data.
- **CLO 1.2 (Understand):** Nắm vững sự tiến hóa ngôn ngữ (Machine $\rightarrow$ Assembly $\rightarrow$ High-Level) và các trụ cột công nghệ đối tượng (Class, Object, Encapsulation, Inheritance).
- **CLO 1.3 (Apply):** Trình bày kiến trúc Microsoft .NET: Common Language Runtime (CLR), Managed Code, MSIL, JIT Compiler và tính độc lập nền tảng (.NET Core / .NET 8).
- **CLO 1.4 (Apply):** Sử dụng thành thạo môi trường Visual Studio IDE: Solution, Project, Form Designer, Toolbox, Properties Window và công cụ gỡ lỗi.
- **CLO 1.5 (Analyze / Create):** Vận dụng mô hình Lập trình trực quan và Hướng sự kiện (Event-Driven) để thực hành chạy thử (Test-Drive) và phân tích ứng dụng Painter.

Phương pháp tiếp cận:

Học tích hợp: **Lý thuyết chuyên sâu** kết hợp **3 bài Hands-on Labs** thực hành tại chỗ trên máy tính.

Tiến trình Bài học 3 Tiết (150 Phút Tích hợp)

TIẾT 1 (50’)

Nền tảng & Dữ liệu

- Phần cứng vs Phần mềm
- Định luật Moore
- 6 khối logic máy tính
- Phân cấp dữ liệu
- Đo lường Byte (KB - YB)
- **[LAB 1]:** Khảo sát CPU, RAM & Số nhị phân

TIẾT 2 (50’)

OOP & Kiến trúc .NET

- Tiến hóa ngôn ngữ
- Compiler vs Interpreter
- Công nghệ đối tượng (OOP)
- Lịch sử ngôn ngữ C#
- Nền tảng Microsoft .NET
- CLR, MSIL & JIT Compiler
- **[LAB 2]:** Biên dịch CLI với .NET SDK

TIẾT 3 (50’)

Visual Studio & GUI Lab

- Hệ điều hành Windows
- Visual Studio IDE
- Solution & Projects
- Mô hình Event-Driven
- Khảo sát điều khiển GUI
- **[LAB 3]:** Test-Drive ứng dụng Painter
- Tổng kết & Dặn dò

Tổng quan Vị thế của Ngôn ngữ C# & Nền tảng .NET

Vì sao chọn C# cho Kỹ sư CNTT Khóa 2026?

- **Thanh lịch & Hiện đại:** Được thiết kế bởi Anders Hejlsberg (cha đẻ Turbo Pascal, Delphi, TypeScript).
- **Mạnh mẽ & Đa năng:** Hỗ trợ đa mô hình lập trình (OOP, Event-Driven, Functional, Generic, Async).
- **Hệ sinh thái khổng lồ:** Phát triển Web (ASP.NET Core), Desktop (WinForms, WPF, MAUI), Mobile (iOS/Android), Cloud (Azure) và Game (Unity 3D).
- **Hiệu năng vượt trội:** .NET 8 / .NET 9 đứng top đầu thế giới về tốc độ xử lý I/O và tối ưu bộ nhớ.

Triết lý Thiết kế C#

”Powerful, Elegant, Safe & Cross-Platform”

Kết hợp sức mạnh xử lý phần cứng của C++ với tính an toàn, năng suất phát triển siêu tốc của Java và Visual Basic.

TIẾT 1: NỀN TẢNG TÍNH TOÁN,
TỔ CHỨC MÁY TÍNH & PHÂN CẤP DỮ LIỆU

1.1 Khái niệm Cốt lõi: Phần cứng & Phần mềm

Phần cứng (Hardware)

- Là các thiết bị vật lý hữu hình cấu thành nên hệ thống máy tính.
- Bao gồm: Bộ vi xử lý (CPU), Bộ nhớ RAM, Ổ đĩa cứng (SSD/HDD), Màn hình, Bàn phím, Chuột, Bo mạch chủ.
- Thực thi trực tiếp các tín hiệu điện tử và lệnh máy ở cấp độ mạch bán dẫn.

Phần mềm (Software)

- Là tập hợp các chỉ thị (Instructions) và chương trình ra lệnh cho phần cứng làm việc.
- **Phần mềm hệ thống:** Hệ điều hành (Windows, Linux, macOS) điều phối tài nguyên.
- **Phần mềm ứng dụng:** Trình duyệt Web, Office, Games, phần mềm doanh nghiệp do lập trình viên xây dựng.

Mối quan hệ cộng sinh:

Phần cứng là *"cơ bắp"* cung cấp năng lực tính toán; Phần mềm là *"bộ não"* dẫn dắt và điều khiển mọi hành vi của phần cứng.

1.2 Máy tính & Internet trong Đời sống & Nghiên cứu

Đột phá công nghệ định hình xã hội hiện đại (Hình 1.1 giáo trình):

- **Hồ sơ y tế điện tử (EHR):** Số hóa toàn bộ bệnh án, đơn thuốc, lịch sử tiêm chủng và phân tích gen; cứu sống hàng triệu bệnh nhân.
- **Hệ thống định vị toàn cầu (GPS):** Mạng lưới vệ tinh tính toán khoảng cách và thời gian truyền tín hiệu, chỉ đường chính xác theo mili-giây.
- **Robot học & Y tế thông minh:** Phẫu thuật xâm lấn tối thiểu với cánh tay robot chính xác từng micromet; robot thăm dò đáy biển và vũ trụ.
- **Thiết bị bay tự hành (Drones/Quadcopters):** Ứng dụng cứu hộ cứu nạn, nông nghiệp thông minh và giao hàng tự động.
- **Xe tự hành (Self-Driving Cars):** Hàng tỷ phép tính mỗi giây xử lý dữ liệu từ Radar, Lidar, Camera để điều khiển phương tiện an toàn.

1.3 Phần cứng: Tốc độ Tính toán & Định luật Moore

Sức mạnh tính toán phi thường:

- Máy tính cá nhân hiện nay thực hiện **hàng tỷ phép tính mỗi giây** (Gigahertz - GHz).
- Siêu máy tính (Supercomputers) đạt đến **hàng triệu tỷ phép tính/giây** (Petaflops, Exaflops).

Định luật Moore (Gordon Moore - 1965)

"Số lượng bóng bán dẫn (transistors) trên một vi mạch tích hợp sẽ tăng gấp đôi xấp xỉ sau mỗi 18 đến 24 tháng, trong khi giá thành không đổi hoặc giảm."

Hệ quả đối với Lập trình viên:

- Phần cứng ngày càng rẻ, mạnh và nhỏ gọn.
- Cho phép chạy các mô hình AI/Machine Learning và phần mềm đồ họa phức tạp trên máy tính cá nhân và smartphone.
- Đòi hỏi lập trình viên phải viết code tối ưu đa luồng (Multi-threading / Async).

1.3.2 Tổ chức Máy tính: 6 Khối Chức năng Logic (Phần 1)

3 Khối đầu tiên (Hình 1.2 Part 1):

1. **Input unit (Khối nhập dữ liệu):**
   - Bộ phận *"tiếp nhận"* thông tin từ bên ngoài vào máy tính.
   - Thiết bị: Bàn phím, chuột, màn hình cảm ứng, micro, máy quét, cảm biến.
2. **Output unit (Khối xuất dữ liệu):**
   - Bộ phận *"chuyển phát"* kết quả xử lý ra người dùng hoặc hệ thống khác.
   - Thiết bị: Màn hình, máy in, loa thanh, bộ điều khiển cơ cấu chấp hành.
3. **Memory unit (Bộ nhớ chính / RAM):**
   - Bộ phận lưu trữ tạm thời tốc độ cực cao.
   - Chứa dữ liệu và lệnh đang được chạy; dữ liệu **mất khi mất điện** (volatile).

Logical unit	Description
Input unit	This "receiving" section obtains information (data and computer programs) from input devices and places it at the disposal of the other units for processing. Most user input is entered into computers through keyboards, touch screens and mouse devices. Other forms of input include receiving voice commands, scanning images and barcodes, reading from secondary storage devices (such as hard drives, DVD drives, Blu-ray Disc™ drives and USB flash drives—also called "thumb drives" or "memory sticks"), receiving video from a webcam and having your computer receive information from the Internet (such as when you stream videos from YouTube® or download e-books from Amazon). Newer forms of input include position data from a GPS device, motion and orientation information from an *accelerometer* (a device that responds to up/down, left/right and forward/backward acceleration) in a smartphone or game controller (such as Microsoft® Kinect® for Xbox®, Wii™ Remote and Sony® PlayStation® Move) and voice input from devices like Amazon Echo and the forthcoming Google Home.
Output unit	This "shipping" section takes information the computer has processed and places it on various output devices to make it available for use outside the computer. Most information that's output from computers today is displayed on screens (including touch screens), printed on paper ("going green" discourages this), played as audio or video on PCs and media players (such as Apple's iPods) and giant screens in sports stadiums, transmitted over the Internet or used to control other devices, such as robots and "intelligent" appliances. Information is also commonly output to secondary storage devices, such as solid-state drives (SSDs), hard drives, DVD drives and USB flash drives. Popular recent forms of output are smartphone and game-controller vibration, virtual reality devices like Oculus Rift and Google Cardboard and mixed reality devices like Microsoft's HoloLens.
Memory unit	This rapid-access, relatively low-capacity "warehouse" section retains information that has been entered through the input unit, making it immediately available for processing when needed. The memory unit also retains processed information until it can be placed on output devices by the output unit. Information in the memory unit is *volatile*—it's typically lost when the computer's power is turned off. The memory unit is often called either memory, primary memory or RAM (Random Access Memory). Main memories on desktop and notebook computers contain as much as 128 GB of RAM, though 2 to 16 GB is most common. GB stands for gigabytes; a gigabyte is approximately one billion bytes. A byte is eight bits. A bit is either a 0 or a 1.
Arithmetic and logic unit (ALU)	This "manufacturing" section performs *calculations*, such as addition, subtraction, multiplication and division. It also contains the *decision* mechanisms that allow the computer, for example, to compare two items from the memory unit to determine whether they're equal. In today's systems, the ALU is implemented as part of the next logical unit, the CPU.

Fig. 1.2 | Logical units of a computer. (Part 1 of 2.)

Hình 1.2: Các khối logic của máy tính (Phần 1/2 – Deitel p.46)

1.3.2 Tổ chức Máy tính: 6 Khối Chức năng Logic (Phần 2)

3 Khối tiếp theo (Hình 1.2 Part 2):

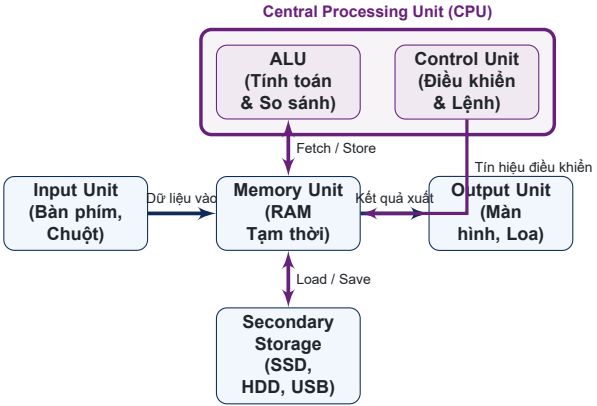
4. **Arithmetic and Logic Unit (ALU):**
   - Bộ phận *"nhà máy"* thực hiện tính toán số học (+, -, *, /) và so sánh logic (=, <, >).
5. **Central Processing Unit (CPU):**
   - *"Bộ não chỉ huy"* điều phối và giám sát toàn bộ hoạt động của tất cả các khối khác trong hệ thống.
   - Hiện nay CPU tích hợp cả ALU và Control Unit trên cùng một chip silicon.
6. **Secondary Storage Unit (Bộ nhớ thứ cấp):**
   - *"Kho chứa hàng"* dài hạn, dung lượng lớn (SSD, HDD, USB).
   - Dữ liệu **vẫn tồn tại khi tắt máy** (persistent / non-volatile).

Logical unit	Description
Central processing unit (CPU)	This "administrative" section coordinates and supervises the operation of the other sections. The CPU tells the input unit when information should be read into the memory unit, tells the ALU when information from the memory unit should be used in calculations and tells the output unit when to send information from the memory unit to certain output devices. Many of today's computers have multiple CPUs and, hence, can perform many operations simultaneously. A multicore processor implements multiple processors on a single integrated-circuit chip—a *dual-core processor* has two CPUs, a *quad-core processor* has four and an *octa-core processor* has eight. Today's desktop computers have processors that can execute billions of instructions per second. Chapter 23 explores how to write apps that can take full advantage of multicore architecture.
Secondary storage unit	This is the long-term, high-capacity "warehousing" section. Programs or data not actively being used by the other units normally are placed on secondary storage devices (e.g., your *hard drive*) until they're again needed, possibly hours, days, months or even years later. Information on secondary storage devices is *persistent*—it's preserved even when the computer's power is turned off. Secondary storage information takes much longer to access than information in primary memory, but its cost per unit is much less. Examples of secondary storage devices include solid-state drives (SSDs), hard drives, DVD drives and USB flash drives, some of which can hold over 2 TB (TB stands for terabytes; a terabyte is approximately one trillion bytes). Typical hard drives on desktop and notebook computers hold up to 2 TB, and some desktop hard drives can hold up to 6 TB.

Fig. 1.2 | Logical units of a computer. (Part 2 of 2.)

Hình 1.2: Các khối logic của máy tính (Phần 2/2 – Deitel p.47)

Sơ đồ Luồng Dữ liệu giữa 6 Khối Kiến trúc Máy tính



Nguyên lý Vàng:

Mọi dữ liệu muốn CPU xử lý **bắt buộc phải được nạp từ Secondary Storage vào RAM**. CPU không làm việc trực tiếp với ổ cứng SSD/HDD vì chênh lệch tốc độ hàng nghìn lần!

1.4 Phân cấp Dữ liệu (Data Hierarchy): Từ Bit đến Big Data

Các cấp độ dữ liệu (Thấp → Cao):

- **Bit (Binary Digit):** Đơn vị nhỏ nhất, mang giá trị `0` hoặc `1`.
- **Byte / Character:** Tập hợp 8 bits; biểu diễn 1 ký tự (mã ASCII, Unicode).
- **Field (Trường dữ liệu):** Nhóm các ký tự mang ý nghĩa (ví dụ: `Ten: "Judy"`).
- **Record (Bản ghi):** Nhóm các trường liên quan của 1 đối tượng (Mã SV, Họ tên, Điểm).
- **File (Tệp):** Tập hợp các bản ghi có cùng cấu trúc.
- **Database (Cơ sở dữ liệu):** Tập hợp nhiều tệp/bảng liên kết chặt chẽ.
- **Big Data:** Dữ liệu siêu lớn, tốc độ cao và đa dạng (Petabytes).

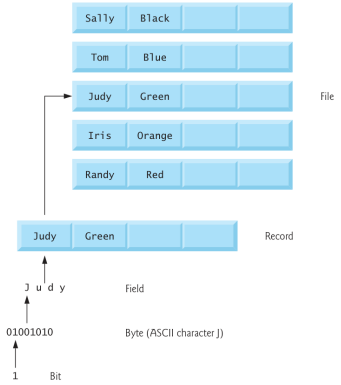


Fig. 1.3 | Data hierarchy.

Hình 1.3: Mô hình phân cấp dữ liệu từ Bit đến File (Deitel p.48)

Bảng Đo lường Dung lượng Dữ liệu: Từ KB đến YB

Quy ước tính toán trong Tin học:

- Vì máy tính dùng hệ nhị phân (cơ số 2), nên các bội số được định nghĩa theo lũy thừa $2^{10} = 1024$:
- 1 KB $= 1024$ Bytes $\approx 10^3$ Bytes
- 1 MB $= 1024$ KB $\approx 10^6$ Bytes
- 1 GB $= 1024$ MB $\approx 10^9$ Bytes
- 1 TB $= 1024$ GB $\approx 10^{12}$ Bytes

Lưu ý thực tế:

Các nhà sản xuất ổ cứng thường tính 1 GB $= 10^9$ bytes, do đó ổ cứng 500 GB hiển thị trong Windows chỉ khoảng 465 GB!

Unit	Bytes	Which is approximately
1 kilobyte (KB)	1024 bytes	10^3 (1024) bytes exactly
1 megabyte (MB)	1024 kilobytes	10^6 (1,000,000) bytes
1 gigabyte (GB)	1024 megabytes	10^9 (1,000,000,000) bytes
1 terabyte (TB)	1024 gigabytes	10^{12} (1,000,000,000,000) bytes
1 petabyte (PB)	1024 terabytes	10^{15} (1,000,000,000,000,000) bytes
1 exabyte (EB)	1024 petabytes	10^{18} (1,000,000,000,000,000,000) bytes
1 zettabyte (ZB)	1024 exabytes	10^{21} (1,000,000,000,000,000,000,000) bytes

Fig. 1.4 | Byte measurements.

Hình 1.4: Bảng các đơn vị đo lường Byte (Deitel p.50)

[CHECKPOINT 1] Kiểm tra Nhanh Kiến thức Tiết 1

Câu hỏi 1: Khối chức năng nào thực hiện phép so sánh logic (A > B)?

A. Input unit
B. Memory unit (RAM)
C. **Arithmetic and Logic Unit (ALU)** *[ĐÁP ÁN ĐÚNG]*
D. Secondary storage unit

Câu hỏi 2: Cấp độ nào trong phân cấp dữ liệu bao gồm nhiều trường (fields) có liên quan?

A. Byte
B. **Record (Bản ghi)** *[ĐÁP ÁN ĐÚNG]*
C. Bit
D. Character

[HANDS-ON LAB 1] Khảo sát Phần cứng & Tính toán Nhị phân

Nhiệm vụ Thực hành (15 Phút tại phòng máy IT26M):

1. Mở **Task Manager** (Ctrl + Shift + Esc) → chọn tab **Performance**:
   - Ghi lại: Tên CPU, Xung nhịp cơ bản (GHz), Số nhân (Cores), Số luồng (Logical Processors).
   - Kiểm tra dung lượng RAM và tốc độ (Speed MHz).
2. Mở **Command Prompt** (CMD) và gõ lệnh xem thông số vi xử lý:

```
C:\> wmic cpu get name, numberofcores, numberoflogicalprocessors, maxclockspeed
Name                                      MaxClockSpeed  NumberOfCores  NumberOfLogicalProcessors
Intel(R) Core(TM) i7-13700H CPU @ 2.40GHz        2400           14             20
```

Bài tập nhị phân tại chỗ:

Chuyển đổi số nguyên 45_{10} sang nhị phân → Đáp án: 101101_2 (gồm 6 bits).

Tổng kết Trọng tâm Tiết 1

3 Khái niệm Cốt lõi Tiết 1

1. **Định luật Moore:** Động lực giúp phần cứng tăng gấp đôi hiệu năng sau mỗi 18-24 tháng.
2. **6 Khối logic:** Input, Output, Memory, ALU, CPU, Secondary Storage.
3. **Phân cấp dữ liệu:** Bit $\rightarrow$ Byte $\rightarrow$ Field $\rightarrow$ Record $\rightarrow$ File $\rightarrow$ Database.

Chuẩn bị bước sang Tiết 2

- Làm thế nào để ra lệnh cho phần cứng máy tính làm việc theo ý muốn?
- Các ngôn ngữ lập trình đã tiến hóa như thế nào?
- Tại sao Microsoft lại phát minh ra C# và nền tảng .NET?

Giải lao 5 phút trước khi vào Tiết 2!

TIẾT 2: TIẾN HÓA NGÔN NGỮ, CÔNG NGHỆ ĐỐI TƯỢNG & KIẾN TRÚC .NET

1.5 Các Thế hệ Ngôn ngữ Lập trình (Language Evolution)

1. Ngôn ngữ máy (Machine Language)

- Là ngôn ngữ **duy nhất** phần cứng CPU có thể hiểu và thực thi trực tiếp.
- Cấu thành từ các chuỗi số nhị phân `0` và `1`.
- Phụ thuộc chặt chẽ vào kiến trúc phần cứng máy tính (Machine-dependent).
- Cực kỳ khó viết, dễ sai sót và không thể bảo trì bởi con người.

2. Hợp ngữ (Assembly Language)

- Sử dụng các từ gợi nhớ tiếng Anh (Mnemonics) viết tắt: `LOAD`, `ADD`, `STORE`.
- Cần chương trình **Assembler** để dịch hợp ngữ sang ngôn ngữ máy.
- Vẫn phụ thuộc vào dòng vi xử lý cụ thể, tốc độ viết code vẫn rất chậm.

3. Ngôn ngữ bậc cao (High-Level Language): C, C++, Java, C#, Python

Cú pháp gần với ngôn ngữ tự nhiên tiếng Anh và ký hiệu toán học; độc lập với phần cứng.

Biên dịch (Compiler) vs Thông dịch (Interpreter)

Trình biên dịch (Compiler)

- Dịch **toàn bộ** mã nguồn bậc cao thành tệp mã máy (Machine code) trước khi thực thi.
- Quá trình biên dịch tốn thời gian ban đầu nhưng chương trình chạy **cực kỳ nhanh**.
- Báo lỗi cú pháp toàn diện trước khi cho phép chạy chương trình.
- *Đại diện:* C, C++, Rust, Go.

Trình thông dịch (Interpreter)

- Đọc và dịch **từng dòng lệnh một**, chạy trực tiếp lệnh đó rồi mới sang dòng kế tiếp.
- Chạy ngay không cần chờ biên dịch, nhưng tốc độ thực thi **chậm hơn đáng kể**.
- Lỗi ở dòng nào chỉ phát hiện khi máy tính chạy đến dòng đó.
- *Đại diện:* Python, JavaScript, Ruby, PHP.

Cơ chế Lai (Hybrid Approach) trong C# / Java:

Biên dịch mã nguồn thành **Mã trung gian (MSIL / Bytecode)**, sau đó dùng **JIT Compiler** trong máy ảo để dịch sang mã máy siêu tốc khi ứng dụng khởi chạy!

1.6 Công nghệ Đối tượng (Object Technology)

Tư duy Hướng đối tượng trong Lập trình:

- Trong thế giới thực, mọi vật xung quanh chúng ta đều là các **Đối tượng (Objects)**: ô tô, điện thoại, con người, tài khoản.
- Mỗi đối tượng đều có hai thành phần:
  1. **Thuộc tính (Attributes / Properties):** Màu sắc, kích thước, số dư tài khoản.
  2. **Hành vi (Behaviors / Methods):** Chạy, dừng, tăng tốc, rút tiền, chuyển khoản.

Mục tiêu Tối thượng của OOP

Tái sử dụng Phần mềm (Software Reuse)

Thay vì tự gõ lại mọi dòng code từ đầu, ta sử dụng các lớp đối tượng có sẵn đạt chuẩn chất lượng cao, giúp tăng năng suất gấp nhiều lần.

Phân biệt Lớp (Class) và Đối tượng (Object)

Lớp (Class) = Bản vẽ thiết kế

- Là khuôn mẫu (Blueprint / Template) định nghĩa cấu trúc dữ liệu và hành vi chung.
- *Ví dụ:* Bản vẽ thiết kế kỹ thuật ô tô trên giấy: chỉ định xe có 4 bánh, vô lăng, màu sơn, động cơ.
- Bản vẽ **không thể tự nổ máy hay chạy trên đường**!

Đối tượng (Object) = Thực thể cụ thể

- Là một thể hiện cụ thể (Instance) được chế tạo từ bản vẽ lớp.
- *Ví dụ:* Chiếc xe VinFast màu đỏ biển số `43A-12345` thực sự lăn bánh ngoài đời.
- Từ 1 Class có thể tạo ra **vô số Objects** với các giá trị thuộc tính riêng biệt!

Cú pháp khởi tạo trong C#:

```
Car myCar = new Car("VinFast VF8", "Do"); // Toán tử new sinh đối tượng trong RAM
```

Các Trụ cột Công nghệ Đối tượng trong C#

1. Đóng gói (Encapsulation)

- Gom nhóm dữ liệu và phương thức xử lý vào trong một đơn vị Class duy nhất.
- **Che giấu thông tin (Information Hiding):** Người lái xe chỉ cần đạp chân ga để tăng tốc, không cần biết chi tiết quá trình bơm nhiên liệu vào buồng đốt!
- Bảo vệ dữ liệu nội bộ không bị can thiệp trái phép từ bên ngoài.

2. Kế thừa (Inheritance)

- Cho phép tạo ra một Class mới dựa trên một Class đã có sẵn.
- Lớp con thừa hưởng toàn bộ thuộc tính và hành vi của lớp cha, đồng thời có thể bổ sung tính năng riêng.
- *Ví dụ:* `XeDien` kế thừa từ `OTo`, bổ sung thêm thuộc tính `DungLuongPin`.

3. Đa hình (Polymorphism):

Cho phép các đối tượng thuộc các lớp khác nhau phản ứng với cùng một thông điệp theo những cách riêng biệt (sẽ học sâu ở Chương 8 & 9).

1.7 Internet và World Wide Web

Các cột mốc tiến hóa mạng thông tin toàn cầu:

- **ARPANET (1969):** Tiền thân của Internet, do Bộ Quốc phòng Mỹ tài trợ, tiên phong kỹ thuật chuyển mạch gói (Packet Switching).
- **Giao thức TCP/IP:** Bộ giao thức mạng nền tảng kết nối các mạng máy tính khác biệt trên thế giới.
- **World Wide Web (1989):** Tim Berners-Lee phát minh siêu văn bản (HTML) và giao thức HTTP, biến Internet thành không gian chia sẻ tài liệu toàn cầu.
- **Web 2.0 (Thập niên 2000):** Chuyển từ web đọc tĩnh sang web tương tác người dùng (Facebook, YouTube, Wikipedia, Cloud SaaS).
- **Điện toán đám mây (Cloud Computing - Microsoft Azure):** Thuê năng lực tính toán và lưu trữ quy mô toàn cầu theo nhu cầu thực tế.
- **Internet of Things (IoT):** Hàng tỷ thiết bị thông minh (đồng hồ, TV, cảm biến xe) kết nối và trao đổi dữ liệu qua Internet.

1.8 Ngôn ngữ C#: Lịch sử Hình thành & Chuẩn hóa

Lịch sử ra đời:

- Được Microsoft công bố vào năm 2000, đi cùng sáng kiến .NET.
- Người đứng đầu nhóm thiết kế: **Anders Hejlsberg** (Kỹ sư trưởng huyền thoại).
- C# được chuẩn hóa quốc tế bởi **ECMA (ECMA-334)** và **ISO/IEC (ISO/IEC 23270)**.

Nguồn gốc tên gọi C#:

Dấu thăng (#) mượn từ âm nhạc (thăng tiến), đồng thời ghép từ 4 dấu cộng (++ của C++ xếp chồng lên nhau thành #), biểu thị sự kế thừa và nâng cấp toàn diện!

Đặc trưng Cốt lõi của C#

- Hướng đối tượng 100% (Pure OOP)
- Định kiểu mạnh mẽ & An toàn (Type-safe)
- Thu gom rác tự động (Garbage Collection)
- Hỗ trợ lập trình trực quan (Visual)
- Hỗ trợ xử lý bất đồng bộ (`async/await`)
- Đa nền tảng (Cross-platform)

Các Phong cách Lập trình Đa dạng trong C#

1. Lập trình Hướng sự kiện (Event-Driven)

- Ứng dụng phản hồi lại các sự kiện (Events) do người dùng kích hoạt: Click chuột, gõ phím, di chuột.
- Cốt lõi của các ứng dụng GUI hiện đại.

2. Lập trình Trực quan (Visual Programming)

- Kéo - thả các điều khiển (Button, TextBox, Grid) trực tiếp trên Form Designer thay vì viết code giao diện thủ công.

3. Lập trình Tổng quát (Generic Programming)

- Viết thuật toán và cấu trúc dữ liệu tổng quát cho mọi kiểu dữ liệu mà vẫn đảm bảo Type-Safety (ví dụ `List<T>`).

4. Lập trình Bất đồng bộ (Async / Await)

- Thực thi các tác vụ tốn thời gian (I/O, gọi API mạng, truy vấn CSDL) mà **không làm đơ / treo giao diện** người dùng.

1.9 Nền tảng Microsoft .NET: Tầm nhìn Kiến trúc

Sáng kiến Microsoft .NET (Ra mắt năm 2000):

Một nền tảng phần mềm toàn diện cho phép phát triển, triển khai và thực thi ứng dụng dễ dàng trên Internet và mạng nội bộ.

1. Thư viện Lớp (.NET FCL)

- Hàng chục nghìn lớp có sẵn được tối ưu hóa hiệu năng cao.
- Cung cấp giải pháp cho: Thao tác tệp (File I/O), Mạng (Networking), CSDL (ADO.NET, Entity Framework), Bảo mật, Giao diện (WinForms, WPF).
- Tái sử dụng tối đa, tiết kiệm 70% thời gian gõ code.

2. Máy ảo CLR (Common Language Runtime)

- Trái tim vận hành của toàn bộ hệ sinh thái .NET.
- Quản lý toàn bộ vòng đời thực thi ứng dụng.
- Ẩn giấu phần cứng và hệ điều hành bên dưới đối với lập trình viên.
- Cung cấp các dịch vụ quản lý tự động: Cấp phát bộ nhớ, thu gom rác, kiểm tra bảo mật.

Khái niệm Managed Code (Mã được Quản lý)

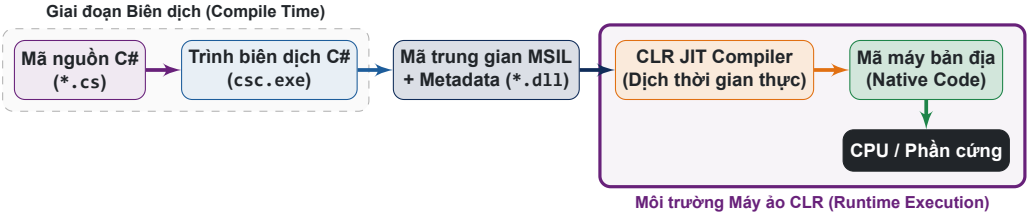
Managed Code (Mã được quản lý)

- Là mã nguồn được biên dịch ra mã trung gian MSIL và **chạy dưới sự giám sát hoàn toàn của CLR**.
- C#, VB.NET, F# mặc định luôn là Managed Code.
- **Lợi ích to lớn:**
  - Không lo rò rỉ bộ nhớ (Memory Leaks).
  - Ngăn chặn lỗi ghi đè vùng nhớ (Buffer Overflow).
  - Bắt lỗi ngoại lệ (Exceptions) an toàn giữa các ngôn ngữ khác nhau.

Unmanaged Code (Mã không quản lý)

- Được biên dịch trực tiếp sang mã máy nhị phân của CPU (ví dụ C, C++ truyền thống).
- Không có máy ảo CLR quản lý.
- Lập trình viên phải tự quản lý từng byte bộ nhớ (`malloc`, `free`, con trỏ).
- Nếu có lỗi con trỏ có thể làm sập toàn bộ hệ điều hành (Crash / BSoD).

Kiến trúc Thực thi .NET: Mã nguồn → MSIL → CLR JIT



Vai trò của MSIL (Microsoft Intermediate Language):

MSIL không phải mã máy của Intel hay ARM. Nhờ đó, tệp `*.dll` có thể mang sang bất kỳ máy tính nào chạy Windows, Linux hay macOS có cài đặt CLR để thực thi mượt mà!

1.9.3 Độc lập Nền tảng (Platform Independence)

Triết lý "Write Once, Run Anywhere":

- Lập trình viên chỉ cần viết và biên dịch code một lần thành MSIL.
- Bất kỳ hệ điều hành nào có máy ảo CLR tương thích đều có thể thực thi chương trình mà **không cần sửa hay biên dịch lại mã nguồn**.
- Tiết kiệm hàng triệu USD chi phí chuyển đổi (porting) phần mềm cho doanh nghiệp.

Sự tiến hóa của .NET

- **.NET Framework (2002 - 2019):** Ban đầu chủ yếu tối ưu hóa cho hệ điều hành Windows.
- **.NET Core (2016):** Viết lại hoàn toàn với mã nguồn mở, hoạt động siêu tốc trên Windows, Linux, macOS.
- **.NET 5 → 8 → 9 (Hiện đại):** Hợp nhất toàn bộ hệ sinh thái .NET thành một nền tảng duy nhất đẳng cấp thế giới.

1.9.4 Khả năng Tương tác Đa ngôn ngữ (Language Interoperability)

Tự do lựa chọn Ngôn ngữ:

- Nền tảng .NET không bắt buộc lập trình viên phải dùng chung một ngôn ngữ.
- Lập trình viên có thể viết Module A bằng **C#**, Module B bằng **Visual Basic**, và Module C bằng **F#**.
- Tất cả đều được biên dịch về cùng một chuẩn **mã trung gian MSIL**!

Sức mạnh Tích hợp Không biên giới:

- Một lớp viết bằng C# có thể **kế thừa trực tiếp** một lớp viết bằng Visual Basic.
- Một phương thức viết bằng C# có thể bắt ngoại lệ (Exception) do mã F# ném ra.
- Toàn bộ thư viện .NET Framework Class Library (FCL) được chia sẻ bình đẳng cho tất cả các ngôn ngữ .NET.

[CHECKPOINT 2] Kiểm tra Nhanh Kiến thức Tiết 2

Câu hỏi 1: Mã nguồn C# khi biên dịch lần đầu sẽ tạo ra loại mã nào?

A. Mã máy nhị phân của CPU Intel x86/x64
B. **Mã trung gian Microsoft Intermediate Language (MSIL / CIL)** *[ĐÁP ÁN ĐÚNG]*
C. Mã hợp ngữ Assembly
D. Mã thông dịch Python bytecode

Câu hỏi 2: Thành phần nào trong CLR dịch MSIL sang mã máy khi ứng dụng chạy?

A. Trình soạn thảo Visual Studio
B. Trình liên kết (Linker)
C. **Trình biên dịch Just-In-Time (JIT Compiler)** *[ĐÁP ÁN ĐÚNG]*
D. Trình dọn rác (Garbage Collector)

[HANDS-ON LAB 2] Trải nghiệm Chu trình Biên dịch .NET qua CLI

Thực hành Biên dịch & Chạy ứng dụng C# Console (10 Phút):

Mở PowerShell hoặc Command Prompt tại máy tính sinh viên và thực hiện 3 lệnh sau:

```
# 1. Kiem tra phien ban .NET SDK da cai dat tren may tinh
C:\> dotnet --version
8.0.204

# 2. Tao mot du an C# Console moi mang ten "HelloIT26M"
C:\> dotnet new console -o HelloIT26M
The template "Console App" was created successfully.

# 3. Di chuyen vao thu muc du an va chay chuong trinh
C:\> cd HelloIT26M
C:\HelloIT26M> dotnet run
Hello, World!
```

Quan sát quan trọng:

Lệnh `dotnet run` tự động gọi trình biên dịch sinh ra tệp `HelloIT26M.dll` (chứa mã MSIL) trong thư mục `bin/Debug/net8.0/`, sau đó kích hoạt CLR để chạy!

TIẾT 3: VISUAL STUDIO IDE &
TEST-DRIVE ỨNG DỤNG ĐỒ HỌA PAINTER (GUI)

1.10 Hệ điều hành Windows & Hệ sinh thái Microsoft

Hệ điều hành Windows:

- Là hệ điều hành máy tính để bàn (Desktop OS) phổ biến nhất thế giới trong nhiều thập kỷ qua.
- Cung cấp các dịch vụ nền tảng:
  - Quản lý bộ nhớ và phân bổ CPU.
  - Đa nhiệm đồng thời (Preemptive Multitasking).
  - Hệ thống tệp tin (NTFS) bảo mật cao.
  - Giao diện người dùng đồ họa (GUI) trực quan.

Tiến trình phát triển (Hình 1.7):

- **Windows 1.0 (1985) $\rightarrow$ 3.1:** Khởi đầu giao diện đồ họa trên nền DOS.
- **Windows 95 / 98 / 2000 / XP:** Kỷ nguyên điện toán 32-bit bùng nổ toàn cầu.
- **Windows 7 / 8 / 10 / 11:** Điện toán 64-bit hiện đại, màn hình cảm ứng, tích hợp dịch vụ Cloud và AI Copilot.

1.11 Môi trường Phát triển Tích hợp: Visual Studio IDE

Visual Studio – IDE số 1 thế giới cho lập trình viên C#:

Tích hợp trọn vẹn toàn bộ công cụ cần thiết cho vòng đời phát triển phần mềm trong một giao diện duy nhất.

Soạn thảo & Thiết kế

- **Code Editor:** Tô màu cú pháp, gợi ý thông minh **IntelliSense**, tự động sửa lỗi.
- **Form Designer:** Thiết kế giao diện trực quan kéo thả.

Quản lý & Thuộc tính

- **Solution Explorer:** Quản lý cây thư mục tệp mã nguồn và thư viện tham chiếu.
- **Properties Window:** Chỉnh sửa kích thước, màu sắc, font chữ của điều khiển.

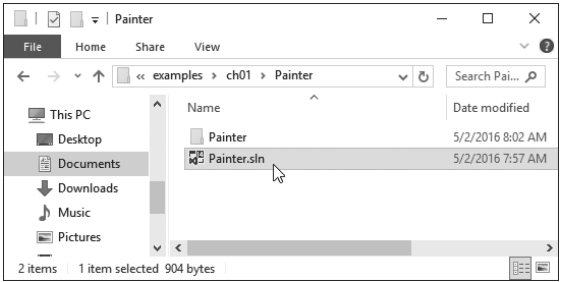
Biên dịch & Debug

- **Build System:** Biên dịch 1-click hoặc phím tắt **F5** / **Ctrl+F5**.
- **Debugger:** Đặt điểm dừng (Breakpoints), soi biến và theo dõi bộ nhớ trực tiếp.

Cấu trúc Solution & Project trong Visual Studio

Mối quan hệ giữa Solution và Project:

- **Solution (`*.sln`):** Là một thùng chứa (Container) ở cấp cao nhất, có thể chứa một hoặc nhiều Project có liên quan với nhau.
- **Project (`*.csproj`):** Chứa toàn bộ các tệp mã nguồn C# (`*.cs`), cấu hình, hình ảnh, tài nguyên để biên dịch thành một tệp thực thi `*.exe` hoặc `*.dll`.



Hình 1.8: Cấu trúc thư mục ứng dụng Painter và tệp Painter.sln (Deitel p.64)

Thao tác chuẩn:

Luôn nhấp đúp vào tệp **`.sln`** để mở toàn bộ dự án trong Visual Studio.

1.12 Case Study: Ứng dụng Đồ họa Painter (Visual C# GUI)

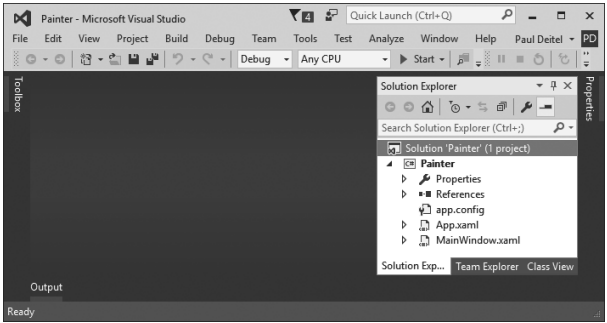
Mô tả Ứng dụng Painter (Deitel Section 1.12):

- Một ứng dụng vẽ hình đồ họa Windows Forms trực quan hoàn chỉnh.
- Người dùng chọn màu cọ vẽ: **Red, Blue, Green, Black**.
- Người dùng chọn kích thước cọ: **Small, Medium, Large**.
- Di chuột trên vùng vẽ màu trắng (**Panel**) để tạo các bức tranh nghệ thuật.
- Cung cấp nút **Undo** (hoàn tác) và **Clear** (xóa sạch vẽ lại).

Bài học Sư phạm cốt lõi

- Giúp sinh viên thấy ngay thành quả trực quan sống động của lập trình C# ngay trong buổi học đầu tiên!
- Không cần viết hàng trăm dòng code đồ họa phức tạp từ đầu; tận dụng các điều khiển GUI có sẵn trong .NET Framework.

Khảo sát các Điều khiển Đồ họa GUI (Hình 1.10)



Hình 1.10: Các thành phần điều khiển GUI trong ứng dụng Painter (Deitel p.65)

4 Loại Điều khiển GUI chính:

1. **GroupBox:** Khung viền nhóm các điều khiển có liên quan với nhau về mặt ngữ nghĩa (nhóm Color, nhóm Size).
2. **RadioButton:** Nút chọn tròn độc quyền; trong cùng 1 GroupBox, chỉ có **duy nhất 1 nút** được chọn tại một thời điểm.
3. **Panel:** Vùng không gian nền trắng tiếp nhận thao tác vẽ và tọa độ chuột.
4. **Button:** Nút bấm kích hoạt hành động tức thì (`Undo`, `Clear`).

Cơ chế Lập trình Hướng sự kiện (Event-Driven Mechanism)

Mô hình Hướng sự kiện hoạt động thế nào?

- Ứng dụng không chạy tuần tự từ dòng đầu đến dòng cuối rồi thoát.
- Ứng dụng rơi vào trạng thái **"Lắng nghe" (Event Loop)**, kiên nhẫn chờ đợi tương tác từ người dùng.
- Khi người dùng tương tác $\rightarrow$ Hệ điều hành gửi thông điệp sự kiện $\rightarrow$ Phương thức xử lý sự kiện (**Event Handler**) tương ứng được kích hoạt.

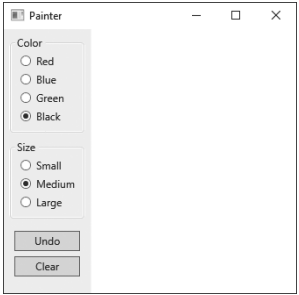
Các sự kiện trong Ứng dụng Painter

- `RadioButton.CheckedChanged`: Đổi màu hoặc kích thước cọ vẽ ngay khi người dùng chọn.
- `Panel.MouseDown`: Nhận biết người dùng bắt đầu nhấn chuột xuống bề mặt vẽ.
- `Panel.MouseMove`: Bắt tọa độ (X, Y) liên tục khi rê chuột để vẽ các hình tròn màu.
- `Button.Click`: Xử lý xóa tranh (`Clear`) hoặc quay lại nét trước (`Undo`).

[HANDS-ON LAB 3] Test-Drive Ứng dụng Painter tại Lớp IT26M

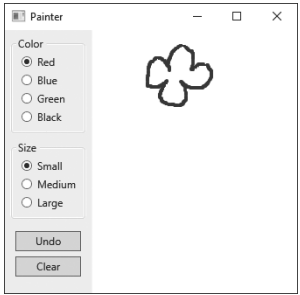
Thực hành Chạy thử & Tương tác 3 Bước (20 Phút tại phòng máy):

Bước 1: Vẽ cánh hoa (Red / Small)



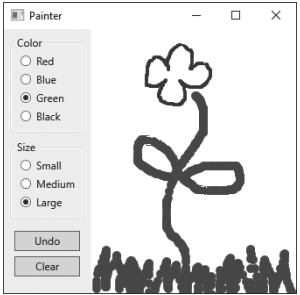
Chọn Red + Small, vẽ các cánh hoa tròn uốn lượn (Hình 1.11)

Bước 2: Vẽ cành & cỏ (Green / Large)



Chọn Green + Large, kéo thẳng vẽ thân hoa và thảm cỏ xanh (Hình 1.12)

Bước 3: Vẽ hạt mưa (Blue / Medium)



Chọn Blue + Medium, chấm các hạt mưa hoàn tất bức tranh (Hình 1.13)

[CHECKPOINT 3] Kiểm tra Nhanh Kiến thức Tiết 3

Câu hỏi 1: Trong ứng dụng Painter, điều khiển nào chứa vùng vẽ màu trắng?

A. GroupBox
B. RadioButton
C. **Panel** *[ĐÁP ÁN ĐÚNG]*
D. Button

Câu hỏi 2: Phím tắt chuẩn trong Visual Studio để biên dịch và chạy ứng dụng là gì?

A. F1
B. **F5 (hoặc Ctrl + F5 để chạy không debug)** *[ĐÁP ÁN ĐÚNG]*
C. F12
D. Ctrl + S

Tổng kết Toàn diện Chương 1: Checklist Chuẩn Đầu Ra

Chúc mừng sinh viên lớp IT26M đã hoàn thành trọn vẹn 5 Chuẩn đầu ra:

- ✓ **CLO 1.1:** Nắm chắc cấu trúc 6 khối logic máy tính và bảng phân cấp dữ liệu từ Bit → Big Data.
- ✓ **CLO 1.2:** Phân biệt Compiler vs Interpreter; thấu hiểu bản chất Công nghệ Đối tượng (Class vs Object, Đóng gói, Kế thừa).
- ✓ **CLO 1.3:** Giải thích thông suốt cơ chế thực thi .NET: Mã nguồn C# → MSIL → CLR JIT → Native Machine Code; hiểu lợi thế của Managed Code.
- ✓ **CLO 1.4:** Làm chủ môi trường Visual Studio IDE (Solution, Project, Form Designer, Toolbox, Properties).
- ✓ **CLO 1.5:** Trải nghiệm thành công mô hình Lập trình trực quan và Hướng sự kiện qua Case Study Painter App.

Nhiệm vụ Tự học & Chuẩn bị cho Buổi học tiếp theo

Bài tập về nhà (Chương 1):

1. Đọc kỹ toàn bộ Chapter 1 trong giáo trình *Visual C# How to Program* (trang 41 – 72).
2. Hoàn thành các câu hỏi **Self-Review Exercises** 1.1 đến 1.29 cuối chương.
3. Tự cài đặt **Visual Studio 2022 Community** (miễn phí) trên laptop cá nhân với workload *".NET desktop development"*.

Đọc trước Chương 2: Nhập môn Cú pháp C#

- Viết ứng dụng C# Console đầu tiên: `Console.WriteLine`.
- Cú pháp hàm `Main`, quy ước đặt tên định danh, chú thích mã nguồn.
- Nhập xuất dữ liệu với `Console.ReadLine`.
- Kiểu dữ liệu nguyên thủy, toán tử số học và cấu trúc rẽ nhánh cơ bản (`if`).

CẢM ƠN CÁC BẠN SINH VIÊN LỚP IT26M!

CHÚC CÁC BẠN HỌC TẬP TỐT VÀ LÀM CHỦ CÔNG NGHỆ C# & .NET

Hỏi đáp & Thảo luận trực tiếp tại giảng đường / Diễn đàn LMS môn học